

Assignment 7: Multithreading and Synchronization

Operating System Lab (Unix)

Problem Statement

Write C programs to demonstrate multithreading using POSIX threads (pthreads) in a Unix/Linux environment. The objective is to understand thread creation, execution, synchronization, and common concurrency issues.

Tasks to Implement

1. Write a program to create multiple threads using `pthread_create()`
2. Write a program where multiple threads perform different tasks concurrently
3. Write a program to demonstrate thread synchronization using mutex (`pthread_mutex_lock`, `unlock`)
4. Write a program to demonstrate race condition and resolve it using mutex
5. Write a program using `pthread_join()` to wait for thread completion
6. Write a program to implement producer-consumer problem using threads
7. Write a program to demonstrate deadlock situation using threads

Instructions

- Use C programming language with POSIX threads
- Compile using `gcc filename.c -o output -lpthread`
- Execute using `./output`
- Use header: `pthread.h`
- Ensure proper synchronization to avoid inconsistent output
- Use `sleep()` where needed to observe behavior

Sample Output

Example (thread creation):

```
Thread 1 is running  
Thread 2 is running  
Thread 3 is running
```

Example (mutex):

```
Counter value: 1  
Counter value: 2  
Counter value: 3  
(Final consistent result)
```

Constraints

- Programs must demonstrate actual concurrent execution
- Avoid undefined behavior due to race conditions
- Ensure proper locking and unlocking of mutex
- Prevent deadlocks where possible (except demonstration task)

Evaluation Criteria

- Correct Implementation of Threads – 40%
- Understanding of Synchronization – 20%
- Handling Race Conditions – 15%
- Code Structure and Clarity – 15%
- Documentation / Comments – 10%

Submission Guidelines

- Submit all programs in a single compressed (.zip) file
- Folder name: RollNo_Assignment7
- Include source files and sample outputs
- Include a README explaining thread behavior

Bonus Tasks (Optional but Recommended)

1. Compare performance of threads vs processes
2. Implement reader-writer problem
3. Use condition variables for synchronization
4. Build a thread pool implementation
5. Implement parallel matrix multiplication